HASTANE RANDEVU SİSTEMİ

Veri Tabanı Sistemleri Dersi – Grup Projesi
Proje Planı ve İş Dağılımı

Ekip Üyesi	Ekip Üyesi	Ekip Üyesi
Veronika	Eray	Sude

1. Proje Hakkında

Bu projede müşterimiz **Doğuş Özel Hastanesi** için hastanenin randevu sürecini yöneten bir veritabanı uygulaması tasarlayıp gerçekleştireceğiz. Hastalar sisteme kaydedilecek, bölümlere (polikliniklere) bağlı doktorlardan randevu alınacak, randevu günü yapılan muayene ve konulan teşhis kaydedilecek, gerekirse hastaya bir veya birden fazla ilaç içeren reçete yazılacak.

Proje iki parçadan oluşuyor: **(1)** PostgreSQL üzerinde, ER diyagramından başlayıp normalleştirilmiş tablolara kadar tasarlanan bir veritabanı; **(2)** bu veritabanına Python ile bağlanan, Tkinter arayüzlü küçük bir masaüstü uygulaması. Sonunda hocanın istediği formatta bir rapor teslim edip projeyi sınıfta sunacağız. Kodlar GitHub'da tutulacak ve **her üye kendi hesabından commit atacak**, çünkü hoca commit geçmişini inceleyecek.

2. Müşteri ve Problem Tanımı

Hocanın istediği gibi projemiz, hayali bir müşterinin gerçek bir problemine çözüm olarak tasarlanmıştır. **Müşterimiz: Doğuş Özel Hastanesi.** Hastane; Kardiyoloji, Dahiliye, Ortopedi, Göz Hastalıkları gibi polikliniklerde hizmet veren orta ölçekli bir özel hastanedir.

Hastanenin bugünkü problemleri:

- Randevular telefonla alınıp kâğıda / dağınık Excel dosyalarına yazılıyor; aynı doktora aynı saatte **çift randevu** verilebiliyor.
- Hastanın geçmiş randevularına, muayenelerine ve teşhislerine hızlıca ulaşılamıyor.
- Doktorlar o günkü hasta listesini önceden göremiyor.
- Reçeteler kâğıt üzerinde kalıyor; hangi ilacın ne kadar yazıldığı takip edilemiyor.
- Yönetim; bölümlerin yoğunluğu, doktor başına hasta sayısı gibi bilgileri raporlayamıyor.

Bizim çözümümüz:

Problem	Sistemdeki çözüm
Çift randevu	Veritabanında kısıt + uygulamada çakışma kontrolü: aynı doktora aynı tarih ve saatte ikinci randevu verilemez
Hasta geçmişine ulaşamama	Hasta, randevu, muayene ve teşhis tabloları ilişkili; tek sorguyla hastanın tüm geçmişi listelenir
Doktorun günlük listesi	Doktor paneli ekranı: seçilen gündeki randevular
Reçete takibi	Reçete ve ilaç tabloları (N–N), doz ve kullanım şekliyle birlikte
Raporlama eksikliği	Bölüm bazında hasta sayısı, en çok yazılan ilaç gibi istatistik sorguları

3. Sistem Neler Yapacak?

- **Hasta işlemleri:** yeni hasta kaydı, hasta bilgilerini güncelleme, TC kimlik no ile arama.
- **Randevu işlemleri:** bölüm ve doktor seçerek randevu alma, aynı doktora aynı saatte ikinci randevu verilmemesi, randevu iptali, hastanın randevu geçmişi.

- **Doktor paneli:** doktorun seçilen gündeki randevularını listeleme, muayene kaydı girme.
- **Muayene ve teşhis:** randevuya bağlı muayene kaydı, muayenede konulan teşhis(ler).
- **Reçete ve ilaç:** muayene sonrası reçete yazma, reçeteye doz ve kullanım şekliyle birden fazla ilaç ekleme.
- **Raporlar / istatistikler:** en çok yazılan ilaçlar, bölüm bazında hasta sayısı, doktor başına muayene sayısı gibi sorgular.

4. Kullanılan Teknolojiler

Teknoloji	Ne için kullanılacak?
PostgreSQL + pgAdmin	Veritabanı sunucusu; tabloların, verilerin ve sorguların yönetimi
Python 3	Uygulamanın programlama dili
psycopg2	Python ile PostgreSQL arasındaki bağlantı kütüphanesi
Tkinter	Masaüstü kullanıcı arayüzü (Python ile birlikte gelir)
draw.io (diagrams.net)	ER diyagramı ve UML çizimleri
Git + GitHub	Ortak kod deposu, sürüm takibi ve commit geçmişi

5. Veritabanı Tabloları (Taslak) ve Sorumlular

Tablo ve sütun isimleri **küçük harfle, Türkçe karakter kullanmadan ve alt çizgiyle** yazılacak (örn. **hasta_id**, **dogum_tarihi**, **randevu_tarihi**).

Tablo	Açıklama	Sorumlu
hasta	Hastanın kimlik ve iletişim bilgileri	Eray
randevu	Hasta–doktor–tarih–saat bilgisi ve randevu durumu (aktif, iptal, tamamlandı)	Eray
bolum	Poliklinikler (Kardiyoloji, Dahiliye vb.)	Sude
doktor	Doktor bilgileri; her doktor bir bölüme bağlı (1–N)	Sude
muayene	Randevu sonucunda yapılan muayene (randevu ile 1–1)	Sude
teshis	Muayenede konulan teşhisler (hastalık adı / kodu)	Sude
ilac	İlaç kataloğu (ad, etken madde, form)	Veronika
recete	Muayene sonucunda yazılan reçete	Veronika
recete_ilac	Reçete ↔ İlaç **N–N ilişkisinin ara tablosu** (doz, kullanım şekli)	Veronika

Not: Tablo listesi ER diyagramı aşamasında birlikte son hâlini alacak; küçük değişiklikler olabilir.

6. Modül Sorumlulukları (Özet)

Veronika	Eray	Sude
• Reçete + İlaç modülü (N–N ilişki) • Veritabanı bağlantı dosyası (db.py) • Ana menü ve ekranların birleştirilmesi • ER diyagramının son hâli • GitHub yönetimi, raporun birleştirilmesi	• Hasta + Randevu modülü • Randevu çakışma kontrolü • İlişkisel şemaya dönüşüm • UML sınıf diyagramı • README ve kurulum talimatı	• Doktor + Bölüm + Muayene + Teşhis modülü • Doktor paneli ekranı • Normalleştirme (1NF–3NF) • UML kullanım durumu şeması • Rapor şablonu ve biçimlendirme

7. Adım Adım İş Dağılımı

Proje 10 adıma bölündü. Her adımda üç kişinin de kendi işi var; böylece hem iş yükü dengeli olur hem de commit geçmişinde herkesin katkısı görünür. Bir adım bitmeden bir sonrakine geçmiyoruz; her adımın sonunda kısa bir kontrol toplantısı yapılır.

Adım	Veronika	Eray	Sude
1. Hazırlık ve GitHub Kurulumu	• GitHub reposunu açar, Eray ve Sude'yi collaborator olarak ekler • Klasör yapısını, .gitignore ve ilk README'yi oluşturur • Commit ve branch kurallarını yazar (Bölüm 10)	• PostgreSQL + pgAdmin kurar, *hastane_randevu* veritabanını oluşturur • Python + psycopg2 kurulumunu test eder • Kurulum adımlarını docs/kurulum.md olarak yazar	• Rapor şablonunu hazırlar (kapak, başlıklar, içindekiler, şekil ve tablo listesi) • Hocanın formatına göre boş bölümleri açar • Kendi bilgisayarında kurulumu yapıp repoyu klonlar
2. Gereksinim Analizi (Rapor: Bölüm 1, 2, 3)	• ilac, recete, recete_ilac için veri gereksinimleri • Tüm fonksiyonel gereksinimlerin birleşik listesi • Kullanıcı rollerinin (hasta, doktor, görevli) belirlenmesi	• hasta, randevu için veri gereksinimleri • Hasta ve randevu fonksiyonel gereksinimleri • Rapor Bölüm 2: Kullanılan Teknolojiler	• bolum, doktor, muayene, teshis için veri gereksinimleri • Doktor paneli ve muayene fonksiyonel gereksinimleri • Rapor Bölüm 1: Giriş (müşteri: Doğuş Özel Hastanesi ve problemleri)
3. Veritabanı Tasarımı (Rapor: Bölüm 4)	• Taslakları birleştirerek ER diyagramını çizer (draw.io) • Varlıklar, nitelikler, ilişkiler ve kardinaliteleri netleştirir • ER diyagramı açıklamasını yazar (4a)	• ER diyagramını dönüştürme kurallarıyla ilişkisel şemaya çevirir (4b) • Birincil ve yabancı anahtarları belirler	• Normalleştirme: 1NF, 2NF, 3NF adımları ve örnekleriyle (4c) • Veritabanı diyagramı – pgAdmin ERD görüntüsü (4d)
4. Tabloların Oluşturulması (Rapor: Bölüm 5)	• ilac, recete, recete_ilac için CREATE TABLE • N–N ara tablosunda bileşik anahtar ve FK'ler • Tüm scriptleri doğru sırayla çalıştıran ana dosya • Bu tablolara örnek veriler (INSERT)	• hasta, randevu için CREATE TABLE • CHECK / UNIQUE kısıtları (örn. aynı doktora aynı saatte tek randevu) • En az 15 hasta ve 25 randevu örnek verisi	• bolum, doktor, muayene, teshis için CREATE TABLE • FK ve NOT NULL kısıtları • Bölüm, doktor ve muayene örnek verileri
5. Sorgular (Rapor: Bölüm 6)	• En çok yazılan ilaçlar (GROUP BY + ORDER BY) • Bir reçetedeki ilaçlar (N–N JOIN) • Alt sorgu (subquery) ve HAVING örneği • Bir VIEW örneği	• Doktorun günlük randevuları • Hastanın randevu geçmişi • Tarih aralığına göre randevular • İptal edilen randevu sayısı	• Bölüm bazında hasta sayısı • Doktor başına muayene sayısı • En sık konulan teşhisler • LEFT JOIN örneği (hiç randevusu olmayan doktorlar)
6. Uygulama Modellemesi (Rapor: Bölüm 7)	• Ekran akışını ve menü yapısını tasarlar • UML diyagramlarını kontrol edip kodla uyumlu hâle getirir	• UML sınıf diyagramı (7b)	• UML kullanım durumu şeması (7a)
7. Python + Tkinter Uygulaması (Rapor: Bölüm 8)	• db.py: bağlantı ve ortak sorgu fonksiyonları • Ana menü (main.py) ve ekranların bağlanması • Reçete yazma ekranı (ilaç seçme, doz girme)	• Hasta kayıt / arama / güncelleme ekranı • Randevu alma ve iptal ekranı (çakışma kontrolüyle)	• Doktor paneli: günlük randevu listesi • Muayene ve teşhis giriş ekranı • Bölüm / doktor listeleme ekranı
8. Test ve Ekran Görüntüleri	• Eray'ın modülünü test eder • Hataları GitHub Issues'a yazar • Bağlantı kodu örneğini rapora ekler (8a)	• Sude'nin modülünü test eder • Kendi ekranlarının görüntüleri ve açıklamaları (8b)	• Veronika'nın modülünü test eder • Kendi ekranlarının görüntüleri ve açıklamaları (8b)
9. Raporun Tamamlanması	• Tüm bölümleri tek raporda birleştirir • Özet bölümünü yazar • Son kontrol	• README'yi tamamlar (kurulum ve çalıştırma) • requirements.txt dosyası • Referanslar bölümü (9)	• İçindekiler, şekil ve tablo listelerini günceller • Yazım ve biçim kontrolü

Adım	Veronika	Eray	Sude
10. Sunum Hazırlığı (Bölüm 9)	• Müşteri, problem ve çözüm slaytları • ER diyagramı ve tasarım slaytları • Reçete ekranı canlı demosu	• Tablolar, ilişkisel şema ve sorgu slaytları • Hasta kaydı + randevu alma canlı demosu	• Slayt şablonu ve tasarımı • Normalleştirme slaytı • Doktor paneli + muayene canlı demosu

8. Rapor Bölümleri – Kim Yazıyor?

Rapor Bölümü	Sorumlu
Kapak, İçindekiler, Şekil Listesi, Tablo Listesi	Sude
i. Özet	Veronika
1- Giriş (müşteri: Doğuş Özel Hastanesi, problemler ve çözüm)	Sude
2- Kullanılan Teknolojiler	Eray
3- Gereksinimler (a. Veri, b. Fonksiyonel)	Herkes kendi tabloları; birleştirme: Veronika
4a- ER Diyagramı	Veronika
4b- İlişkisel Şema	Eray
4c- Normalleştirme / 4d- Veritabanı Diyagramı	Sude
5- Veritabanı Uygulama (CREATE TABLE)	Herkes kendi tabloları
6- Sorgular	Herkes kendi sorguları (kişi başı en az 4)
7a- UML Kullanım Durumu / 7b- UML Sınıf Diyagramı	Sude / Eray
8a- Bağlantı Kodu Örneği	Veronika
8b- Kullanıcı Arayüzleri	Herkes kendi ekranları
9- Referanslar	Eray

9. Proje Sunumu

Hoca projeyi hem rapora hem de **sunumda sorulan sorulara verdiğimiz cevaplara** göre değerlendirecek. Bu yüzden sunumu üçümüz birlikte yapacağız ve **herkes sadece kendi kısmını değil, projenin tamamını** (özellikle ER diyagramını, tablolar arası ilişkileri ve sorguları) anlatabilecek durumda olacak.

Sıra	Sunum Bölümü	Sunan
1	Müşteri ve problem: Doğuş Özel Hastanesi neden böyle bir sisteme ihtiyaç duyuyor?	Veronika
2	Gereksinimler ve ER diyagramı (varlıklar, ilişkiler, N–N örneği)	Veronika
3	İlişkisel şema ve tablolar (birincil / yabancı anahtarlar, kısıtlar)	Eray
4	Normalleştirme (1NF, 2NF, 3NF)	Sude
5	Örnek sorgular ve sonuçları	Eray
6	Canlı demo: hasta kaydı ve randevu alma (çift randevu engeli)	Eray
7	Canlı demo: doktor paneli, muayene ve teşhis girişi	Sude
8	Canlı demo: reçete yazma ve istatistikler	Veronika
9	Sonuç ve soru–cevap	Hep birlikte

Soru–cevap için hazırlık:

- Sunumdan önce en az bir prova yapılır; herkes birbirine olası hoca soruları sorar.

- Hazırlanacak tipik sorular: *"Reçete ile ilaç arasındaki ilişki neden N–N?"*, *"Muayene ile randevu neden 1–1?"*, *"Bu tablo hangi normal formda, neden?"*, *"Çift randevuyu nasıl engelliyorsunuz?"*, *"Python veritabanına nasıl bağlanıyor?"*
- Demo öncesi veritabanında yeterli örnek veri olduğu ve uygulamanın sorunsuz açıldığı kontrol edilir.

10. GitHub ve Commit Kuralları

- Herkes **kendi GitHub hesabıyla** ve kendi bilgisayarından commit atar. Başkasının yerine commit atılmaz.
- Küçük ve sık commit: her commit tek bir anlamlı iş içersin (örn. *"hasta tablosu için CREATE TABLE eklendi"*, *"randevu çakışma kontrolü eklendi"*).
- *"update"*, *"düzeltme"*, *"asdf"* gibi anlamsız commit mesajları kullanılmaz.
- Çalışmaya başlamadan önce **git pull**, bitirince **git push**. Büyük işler kendi branch'inde yapılıp Pull Request ile birleştirilir; PR'ı Veronika onaylar.
- Veritabanı şifresi gibi kişisel bilgiler repoya yüklenmez (config dosyası .gitignore'da).

Önerilen klasör yapısı:

hastane-randevu-sistemi/	Ana klasör (GitHub reposu)
sql/	01_create_tables.sql, 02_insert_data.sql, 03_queries.sql
app/	main.py, db.py, hasta_ekrani.py, randevu_ekrani.py, doktor_paneli.py, recete_ekrani.py
docs/	er_diyagrami.png, uml/, ekran_goruntuleri/, kurulum.md
rapor/	proje_raporu.docx / .pdf
requirements.txt	Gerekli Python kütüphaneleri (psycopg2)
README.md	Proje tanıtımı, kurulum ve çalıştırma